

Módulo 5 — Repetição (Loops)

Jornada DEV START · Aula 5 · 17/07 (sexta) · Programa START — TOTVS Paulista

Material entregue ao final da Aula 5. Guarde para consultar sempre que precisar.



Onde você está na jornada

```
1 → 2 → 3 → 4 → [5] → 6 | 7 → 8 → 9 → 10
Fundamentos (Harbour) | Protheus (ADVPL)
▲ você está aqui
```

Você já sabe **guardar** dados (Módulo 3) e fazer o programa **decidir** caminhos (Módulo 4). Falta a terceira grande ferramenta: **repetir**. Hoje você aprende a mandar o computador executar a mesma tarefa dezenas, milhares de vezes — sem reescrever uma linha. É aqui que a máquina mostra sua maior força. 



Roteiro da aula (2h)

Bloco	Tempo	O quê
Retomada	~10 min	Revisão do Módulo 4 (IF , DO CASE) + correção de exercício-chave
Conteúdo	~60 min	WHILE , loop infinito, contador/acumulador, FOR/NEXT , EXIT / LOOP
Mão na massa	~40 min	Construir juntos uma tabuada interativa que repete quantas vezes o usuário quiser
Q&A / networking	~10 min	Dúvidas abertas e integração da turma

Objetivos do módulo

Ao final desta aula você será capaz de: - Usar `WHILE/ENDDO` para repetições com **pré-teste** - Usar `FOR/NEXT` para **laços contados** (com `STEP`) - Controlar laços com `EXIT` e `LOOP` - **Reconhecer e evitar o loop infinito** - Usar laços encadeados e loops de **consistência de entrada** - Entender os conceitos de **contador, acumulador e termo**

Retomando a aula anterior

No **Módulo 4** você aprendeu a fazer o programa **decidir** com `IF/ELSEIF/ELSE` e `DO CASE`, sempre a partir de uma **condição** (`.T.` / `.F.`).

```
IF nSaldo >= nValor
    QOut("Pagamento autorizado")
ELSE
    QOut("Saldo insuficiente")
ENDIF
```

Guarde bem a ideia de **condição** — porque um laço é, em essência, uma **decisão que se repete**: “*enquanto* esta condição for verdadeira, faça de novo”. A mesma pergunta `.T.` / `.F.` que decidia um caminho agora decide se o bloco **repete ou para**.

5.1 Por que precisamos de repetição?

Imagine calcular a soma de 1000 números digitados pelo usuário. Escrever 1000 linhas de `ACCEPT ... TO nX` seria absurdo. A solução são as **estruturas de repetição** (também chamadas de laços, loops ou iterações).

A ideia é simples: você escreve o bloco de código **uma vez**, e o computador o executa quantas vezes for necessário.



Analogia: é como uma esteira de bagagens no aeroporto. A esteira (o laço) roda o tempo todo; a cada volta, uma mala nova (um novo dado) passa pelo mesmo trecho e recebe o mesmo tratamento. Você monta o trecho **uma vez**.

Prova matemática (**Böhm e Jacopini, 1966**): apenas **três estruturas de controle** bastam para resolver qualquer problema computável: 1. **Sequência** (uma instrução após a outra) 2. **Seleção** (`IF` — Módulo 4) 3. **Repetição** (`WHILE` — este módulo)

Você está prestes a completar o trio. 🎯

 Fluxograma do laço WHILE com pré-teste: testa a condição e, se verdadeira, executa e volta a testar

5.2 WHILE — Laço com Pré-teste

A estrutura fundamental de repetição. Executa o bloco **enquanto** a condição for verdadeira.

Sintaxe

```
WHILE <condição>  
    // bloco executado enquanto a condição = .T.  
ENDDO
```

Pré-teste: a condição é verificada **antes** de cada execução. Se já começar falsa, o bloco **nunca** executa.

Exemplo básico: contar de 1 a 5

```
FUNCTION Main()  
    LOCAL nCont := 1  
  
    WHILE nCont <= 5  
        QOut("Contagem: " + Str(nCont))  
        nCont++    // incrementa o contador  
    ENDDO  
  
    QOut("Fim!")  
  
    RETURN NIL
```

Saída:

```
Contagem: 1  
Contagem: 2  
Contagem: 3  
Contagem: 4  
Contagem: 5  
Fim!
```

Os três ingredientes de todo **WHILE** saudável: 1. **Inicialização** antes do laço (`nCont := 1`) 2. **Condição** de parada (`nCont <= 5`) 3. **Atualização** dentro do laço (`nCont++`) — é ela que faz a condição um dia virar falsa

Exemplo: loop com condição de saída

```
FUNCTION Main()
  LOCAL cResposta := "S"
  LOCAL nX, nR

  WHILE Upper(cResposta) == "S"
    ACCEPT "Valor de X: " T0 nX
    nX := Val(nX)
    nR := nX * 3
    QOut("X x 3 = " + Str(nR))
    ACCEPT "Continuar? (S/N): " T0 cResposta
  ENDDO

  RETURN NIL
```

⚠ Loop infinito — o erro clássico

O erro mais comum com laços é **esquecer de atualizar a condição** dentro do bloco. Sem a atualização, a condição nunca vira falsa e o programa **trava repetindo para sempre**.

```
// ERRADO — loop infinito!
LOCAL nCont := 1
WHILE nCont <= 5
  QOut(Str(nCont))
  // Faltou: nCont++    ← a condição nunca muda!
ENDDO
```

🛑 **Regra de ouro:** todo **WHILE** precisa ter, dentro dele, algo que **caminhe em direção à parada**. Ao escrever um laço, pergunte-se: *“o que aqui vai, um dia, tornar esta condição falsa?”* Se você não souber responder, tem um loop infinito.

Travou um loop infinito no terminal? Interrompa com **Ctrl + C**.

 Existe um uso **intencional** de “loop infinito”: o `WHILE .T.` combinado com `EXIT`, que você verá nas seções 5.6 e 5.7. Nesse caso a saída é controlada por dentro — de propósito.

5.3 Conceitos: Contador, Acumulador e Termo

Esses três conceitos aparecem em quase todo problema com loops.

Contador

Variável que **conta** quantas iterações ocorreram. Geralmente incrementada de 1 em 1.

```
LOCAL nCont := 0
WHILE nCont < 10
    nCont++
    // ...
ENDDO
QOut("Total de iterações: " + Str(nCont))
```

Acumulador

Variável que **soma (acumula)** valores ao longo das iterações. **Sempre inicializada antes do loop** (senão você acumula “lixo”).

```

FUNCTION Main()
    LOCAL nTotal := 0      // acumulador – inicializado FORA do loop
    LOCAL nQtd      // quantos números o usuário quer somar
    LOCAL nNum
    LOCAL nI      := 1      // contador do laço

    ACCEPT "Quantos números? " TO nQtd
    nQtd := Val(nQtd)

    WHILE nI <= nQtd
        ACCEPT "Número " + Str(nI) + ": " TO nNum
        nTotal += Val(nNum)    // acumula
        nI++
    ENDDO

    QOut("Soma total: " + Str(nTotal))
    QOut("Média: " + Str(nTotal / nQtd, 8, 2))

    RETURN NIL

```

Termo

Variável cujo valor **muda a cada iteração** para compor uma série.

```

// Série de Fibonacci – os 10 primeiros termos
FUNCTION Main()
    LOCAL nA := 0
    LOCAL nB := 1
    LOCAL nTermo
    LOCAL nI := 1

    WHILE nI <= 10
        nTermo := nA + nB
        QOut(Str(nTermo))
        nA := nB
        nB := nTermo
        nI++
    ENDDO

    RETURN NIL

```

5.4 DO WHILE — nota sobre pós-teste

Existe também a forma `DO WHILE <condição> ... ENDDO`.

```
DO WHILE <condição>
    // bloco executado enquanto a condição = .T.
ENDDO
```

Na prática, em Harbour o `WHILE` e o `DO WHILE` têm o **mesmo comportamento** (pré-teste). Use `WHILE` como padrão. Quando você precisar de um **pós-teste** de verdade (executar ao menos uma vez e só então testar), monte-o com `WHILE .T.` + `EXIT` (próxima seção).

5.5 FOR/NEXT — Laço Contado

Use quando você sabe antecipadamente **quantas vezes** o loop deve executar. O `FOR` cuida do contador para você: inicializa, testa e incrementa automaticamente.

Sintaxe

```
FOR <variável> := <início> TO <fim> [STEP <passo>]
    // bloco executado
NEXT
```

- O `STEP` define o incremento (padrão = 1).
- A variável contadora é controlada **automaticamente**.
- Aceita **decremento** com `STEP` negativo.

Exemplo: tabuada

```
FUNCTION Main()
    LOCAL nNum, nI

    ACCEPT "Tabuada de qual número? " TO nNum
    nNum := Val(nNum)

    QOut("=== Tabuada do " + Str(Int(nNum)) + " ===")
    FOR nI := 1 TO 10
        QOut(Str(nNum) + " x " + Str(nI) + " = " + Str(nNum * nI))
    NEXT

    RETURN NIL
```

Exemplo: ordem decrescente

```
FOR nI := 10 TO 1 STEP -1
    QOut(Str(nI))
NEXT
// Exibe: 10, 9, 8, 7, 6, 5, 4, 3, 2, 1
```

Exemplo: somar os números pares de 1 a 100

```
LOCAL nSoma := 0
FOR nI := 2 TO 100 STEP 2
    nSoma += nI
NEXT
QOut("Soma dos pares de 1 a 100: " + Str(nSoma))
// Resultado: 2550
```

 Comparação FOR/NEXT contado versus WHILE por condição

5.6 EXIT e LOOP — controle de fluxo dentro do laço

EXIT

Sai imediatamente do laço, independentemente da condição.

```
// Buscar um número em uma sequência
LOCAL nI
LOCAL lEncontrou := .F.
LOCAL nBusca := 7

FOR nI := 1 TO 100
    IF nI == nBusca
        lEncontrou := .T.
        EXIT      // para de procurar – já achei
    ENDIF
NEXT

IF lEncontrou
    QOut("Encontrei o número " + Str(nBusca) + " na posição " + Str(nI))
ENDIF
```

LOOP

Volta para o início do laço, pulando o restante do bloco atual (segue para a próxima iteração).


```
// Imprimir apenas os números ímpares de 1 a 20
FOR nI := 1 TO 20
  IF Mod(nI, 2) == 0
    LOOP      // número par – pula para o próximo
  ENDIF
  QOut(Str(nI)) // só chega aqui se for ímpar
NEXT
```

Comando	O que faz
EXIT	Abandona o laço de vez e continua depois dele
LOOP	Interrompe só a iteração atual e vai para a próxima

5.7 Consistência de Entrada

Um princípio fundamental: **nunca confie que o usuário vai digitar o valor correto.**

O padrão é um loop que fica pedindo o dado **até** receber um valor válido:

```
FUNCTION Main()
  LOCAL nNota
  LOCAL cInput

  // Loop de consistência para a nota (0 a 10)
  WHILE .T.
    ACCEPT "Digite a nota (0 a 10): " TO cInput
    nNota := Val(cInput)
    IF nNota >= 0 .AND. nNota <= 10
      EXIT // valor válido – sai do loop
    ENDIF
    QOut("Nota inválida! Digite um valor entre 0 e 10.")
  ENDDO

  QOut("Nota registrada: " + Str(nNota, 5, 1))

RETURN NIL
```

Exemplo mais completo: validação de senha (com limite de tentativas)

```
FUNCTION Main()
  LOCAL cSenhaCorreta := "1234"
  LOCAL cSenhaDigitada
  LOCAL nTentativas := 0
  LOCAL nMaxTentativas := 3

  WHILE nTentativas < nMaxTentativas
    ACCEPT "Digite a senha: " TO cSenhaDigitada
    nTentativas++

    IF cSenhaDigitada == cSenhaCorreta
      QOut("Acesso autorizado!")
      RETURN NIL
    ELSE
      QOut("Senha inválida! Tentativa " + Str(nTentativas) + " de " +
        Str(nMaxTentativas))
    ENDIF
  ENDDO

  QOut("Limite de tentativas atingido. Sistema bloqueado.")

  RETURN NIL
```

Repare que aqui há **duas** formas de sair: acertar a senha (**RETURN NIL**) ou estourar o número de tentativas (a condição **nTentativas < nMaxTentativas** vira falsa).

5.8 Laços Encadeados

Um laço **dentro** de outro. Cada iteração do laço externo executa **todas** as iterações do laço interno.

```
// Tabuada completa de 1 a 10
FUNCTION Main()
  LOCAL nI, nJ

  FOR nI := 1 TO 10
    QOut("")
    QOut("=== Tabuada do " + Str(nI) + " ===")
    FOR nJ := 1 TO 10
      QOut(Str(nI) + " x " + Str(nJ) + " = " + Str(nI * nJ))
    NEXT
  NEXT

  RETURN NIL
```

Contagem total de iterações: $10 \times 10 = 100$ execuções do bloco interno.

Exemplo: padrão de asteriscos

```
// Imprimir um triângulo de asteriscos
FUNCTION Main()
  LOCAL nI, nJ, cLinha

  FOR nI := 1 TO 5
    cLinha := ""
    FOR nJ := 1 TO nI
      cLinha += "*"
    NEXT
    QOut(cLinha)
  NEXT

  RETURN NIL
// Saída:
// *
// **
// ***
// ****
// *****
```

5.9 Loop Geral de Programa

Um padrão muito usado: o programa principal roda dentro de um loop que só encerra quando o usuário pede para sair. É o **WHILE .T.** unindo tudo o que você aprendeu — laço + **DO CASE** .

```

FUNCTION Main()
  LOCAL cOpcao

  WHILE .T.
    QOut("")
    QOut("=== MENU PRINCIPAL ===")
    QOut("1 - Calcular IMC")
    QOut("2 - Calcular salário")
    QOut("0 - Sair")
    ACCEPT "Opção: " TO cOpcao

    DO CASE
      CASE cOpcao == "1"
        CalcularIMC()
      CASE cOpcao == "2"
        CalcularSalario()
      CASE cOpcao == "0"
        QOut("Até logo!")
        EXIT
      OTHERWISE
        QOut("Opção inválida!")
    ENDCASE
  ENDDO

  RETURN NIL

```

📖 Esse “menu que só sai no 0” é a espinha dorsal de praticamente todo sistema de terminal — e a base de como as rotinas do Protheus se organizam. Guarde bem este padrão.

5.10 WHILE vs FOR — quando usar cada um?

Situação	Use
Sabe quantas repetições (1 a N)	FOR/NEXT
Não sabe quantas repetições	WHILE/ENDDO
Loop até o usuário decidir parar	WHILE .T. + EXIT
Loop de validação de entrada	WHILE .T. + EXIT
Percorrer arrays	FOR com índice (ou AEVAL(), no Módulo 6)

Regra prática: contagem conhecida → FOR . Parada por condição/evento → WHILE .



Mão na massa (em aula)

Vamos construir juntos uma **tabuada interativa** que combina **FOR**, **WHILE** e consistência:

Crie um arquivo **tabuada.prg** que: 1. Rode dentro de um **WHILE .T.** (o “loop geral”). 2. Peça um número ao usuário. Se for **0**, encerre com **EXIT**. 3. Use um **FOR nI := 1 TO 10** para imprimir a tabuada do número escolhido. 4. Ao terminar, pergunte “**Quer outra tabuada? (S/N)**” e, se a resposta **não** for **S**, saia. 5. **Incremento (se der tempo)**: valide que o número digitado é positivo antes de gerar a tabuada (loop de consistência).

Faremos passo a passo — e no fim cada um roda a sua tabuada quantas vezes quiser. ✕



Referência rápida do Módulo 5

```
// Laço com pré-teste (não sabe quantas vezes)
WHILE <condição>
    // ...          ← lembre de ATUALIZAR a condição (senão: loop infinito!)
ENDDO

// Laço contado (sabe quantas vezes)
FOR nI := <início> TO <fim> STEP <passo>
    // ...
NEXT

// Controle de fluxo dentro do laço
EXIT      // sai do laço imediatamente
LOOP      // pula para a próxima iteração

// Loop geral / validação de entrada
WHILE .T.
    // ...
    IF <condição de saída>
        EXIT
    ENDIF
ENDDO

// Contador / Acumulador
nCont++      // conta (+1)
nTotal += nValor // acumula (soma no total)
Mod(nI, 2) == 0 // nI é par?
```



Exercícios

Os exercícios desta aula estão em `exercicios.md` (nesta mesma pasta). **Prazo sugerido:** até a **Aula 6 (20/07)**. Há um **fim de semana pela frente** — ótima janela para praticar com calma e consolidar os fundamentos. 💪



Para a próxima aula

Módulo 6 — Funções e Arrays. Até aqui você escreveu tudo dentro de `Main()`. Na próxima aula vamos **organizar o código em pedaços reutilizáveis** (funções) e aprender a guardar **muitos dados juntos** em **arrays** — exatamente o que faltava para os loops brilharem (percorrer listas!).



No fim de semana: rode o seu `tabuada.py` e tente o **desafio do jogo “adivinha o número”** (exercício 6). Dúvidas? Discord `#duvidas-parte1-harbour`. Nos vemos segunda! 🚀